



Basic Plotly Charts

Estimated time needed: 45 minutes

Objectives

After completing this lab, you will be able to:

- Use plotly graph objects and plotly express libraries to plot different types of charts
- Create interesting visualizations on Airline Reporting Carrier On-Time Performance Dataset

Plotly graph objects and Plotly express libraries to plot different types of charts

Plotly Libraries

plotly.graph_objects: This is a low level interface to figures, traces and layout. The Plotly graph objects module provides an automatically generated hierarchy of classes (figures, traces, and layout) called graph objects. These graph objects represent figures with a top-level class `plotly.graph_objects.Figure`.

plotly.express: Plotly express is a high-level wrapper for Plotly. It is a recommended starting point for creating the most common figures provided by Plotly using a simpler syntax. It uses graph objects internally. Now let us use these libraries to plot some charts We will start with `plotly_graph_objects` to plot line and scatter plots

Note: You can hover the mouse over the charts whenever you want to view any statistics in the visualization charts

Exercise I: Get Started with Different Chart types in Plotly

```
In [1]: # Import required libraries
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import numpy as np
```

1. Scatter Plot:

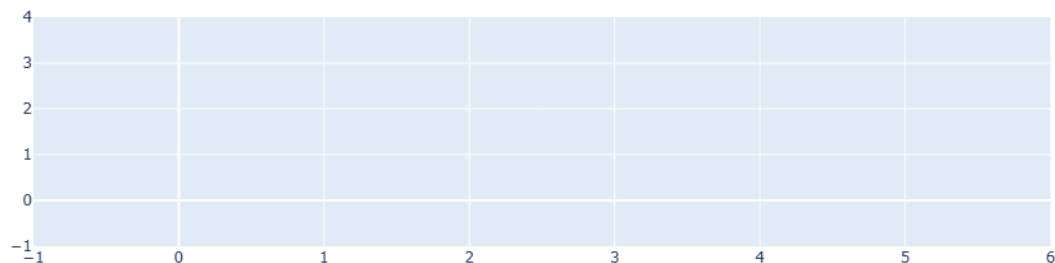
A scatter plot shows the relationship between 2 variables on the x and y-axis. The data points here appear scattered when plotted on a two-dimensional plane. Using scatter plots, we can create exciting visualizations to express various relationships, such as:

- Height vs weight of persons
- Engine size vs automobile price
- Exercise time vs Body Fat

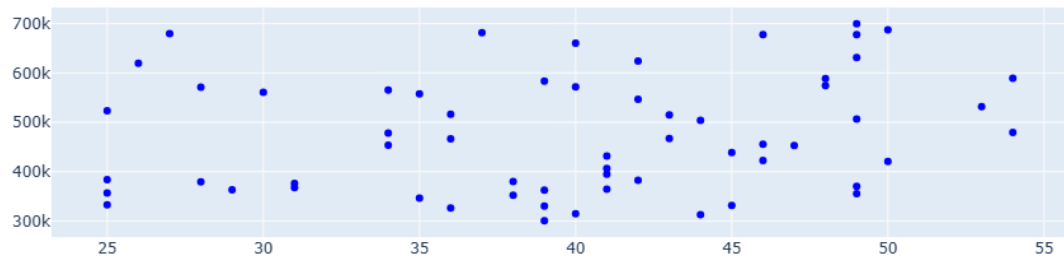
```
In [2]: ##Example 1: Let us illustrate the income vs age of people in a scatter plot
age_array=np.random.randint(25,55,60)

# Define an array containing salesamount values
income_array=np.random.randint(300000,700000,3000000)
```

```
In [3]: ##First we will create an empty figure using go.Figure()
fig=go.Figure()
fig
```



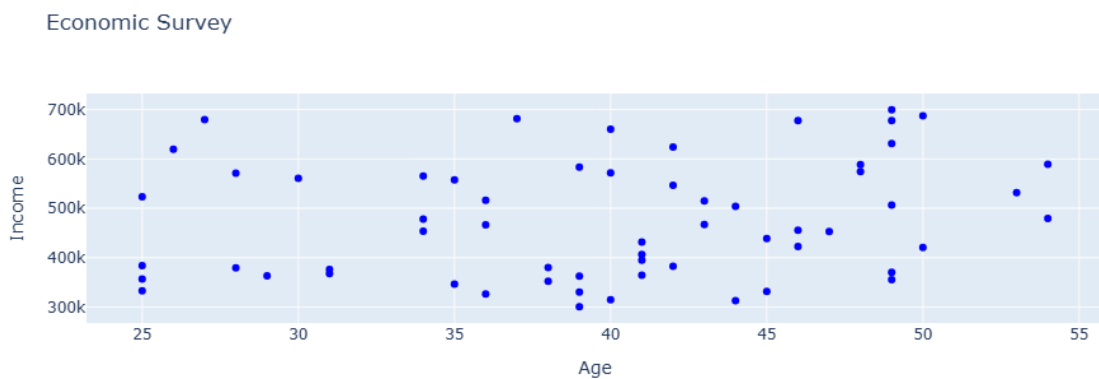
```
In [4]: #Next we will create a scatter plot by using the add_trace function and use the go.
# In go.Scatter we define the x-axis data,y-axis data and define the mode as marker
fig.add_trace(go.Scatter(x=age_array, y=income_array, mode='markers', marker=dict(c
```



However in the previous output title, x-axis and y-axis labels are missing. Let us use the `update_layout` function to update the title and labels.

```
In [5]: ## Here we update these values under function attributes such as title,xaxis_title
fig.update_layout(title='Economic Survey', xaxis_title='Age', yaxis_title='Income')

# Display the figure
fig.show()
```



Inferences:

From the above plot we find that the Income of a person is not correlated with age. We find that as the age increases the income may or not decrease.

2. Line Plot:

A line plot shows information that changes continuously with time. Here the data points are connected by straight lines. Line plots are also plotted on a two dimensional plane like scatter plots. Using line plots, we can create exciting visualizations to illustrate:

- Annual revenue growth

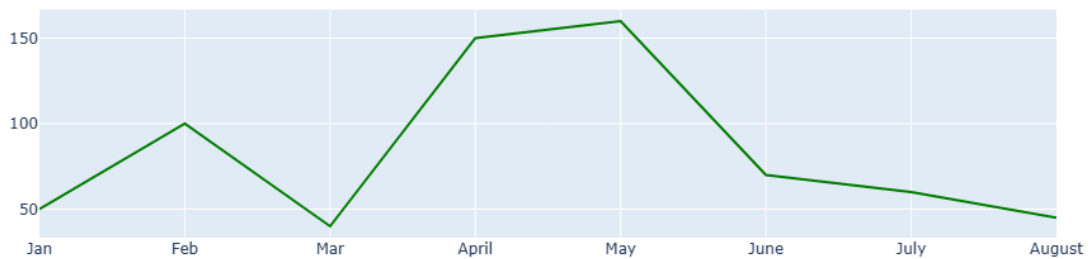
- Stock Market analysis over time
- Product Sales over time

```
In [6]: ##Example 2: Let us illustrate the sales of bicycles from Jan to August last year u
# Define an array containing numberofbicyclesold
numberofbicyclesold_array=[50,100,40,150,160,70,60,45]

# Define an array containing months
months_array=["Jan", "Feb", "Mar", "April", "May", "June", "July", "August"]
```

```
In [7]: ##First we will create an empty figure using go.Figure()
fig=go.Figure()
```

```
In [8]: #Next we will create a line plot by using the add_trace function and use the go.sca
# In go.Scatter we define the x-axis data,y-axis data and define the mode as lines
fig.add_trace(go.Scatter(x=months_array, y=numberofbicyclesold_array, mode='lines'))
```



```
In [9]: ## Here we update these values under function attributes such as title,xaxis_title
fig.update_layout(title='Bicycle Sales', xaxis_title='Months', yaxis_title='Number

# Display the figure
fig.show()
```



Inferences:

From the above plot we find that the sales is the highest in the month of May and then there is a decline in sales.

We will now use plotly express library to plot the other graphs

3.Bar Plot:

A bar plot represents categorical data in rectangular bars. Each category is defined on one axis, and the value counts for this category are represented on another axis. Bar charts are generally used to compare values. We can use bar plots in visualizing:

- Pizza delivery time in peak and non peak hours
- Population comparison by gender
- Number of views by movie name

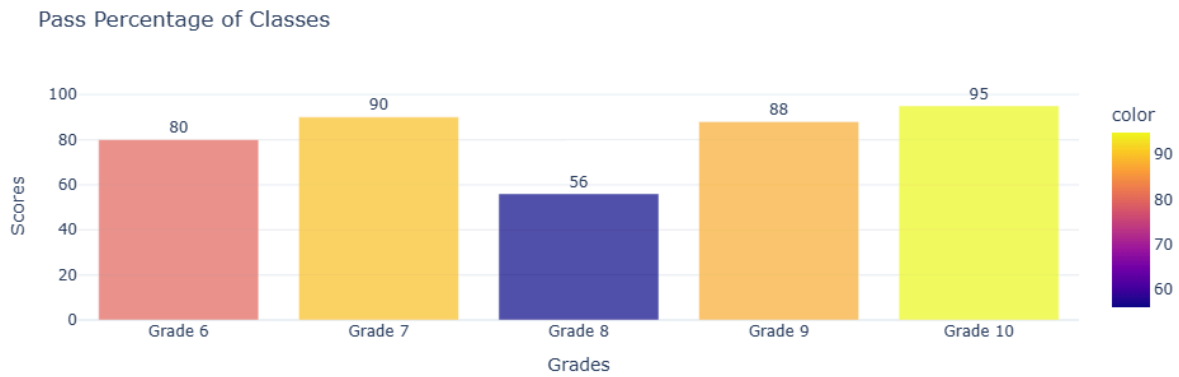
```
In [10]: ##Example 3: Let us illustrate the average pass percentage of classes from grade 6
# Define an array containing scores of students
score_array=[80,90,56,88,95]

# Define an array containing Grade names
grade_array=['Grade 6','Grade 7','Grade 8','Grade 9','Grade 10']
```

In plotly express we set the axis values and the title within the same function call `px.`

`<graphtype>(x=<xaxis value source>,y=<y-axis value source>,title=<appropriate title as a string>)`. In the below code we use `px.bar(x=grade_array, y=score_array, title='Pass Percentage of Classes')`.

```
In [11]: # Use plotly express bar chart function px.bar. Provide input data, x and y axis var
# This will give average pass percentage per class
fig = px.bar(x=grade_array,
             y=score_array,
             color=score_array,
             text=score_array,
             title='Pass Percentage of Classes',
             opacity=0.7,
             labels=dict(x="Grades", y="Scores"),
             template="plotly_white"
            )
fig.update_traces(textposition="outside", cliponaxis=False)
fig.show()
```



Inferences:

From the above plot we find that Grade 8 has the lowest pass percentage and Grade 10 has the highest pass percentage

4.Histogram:

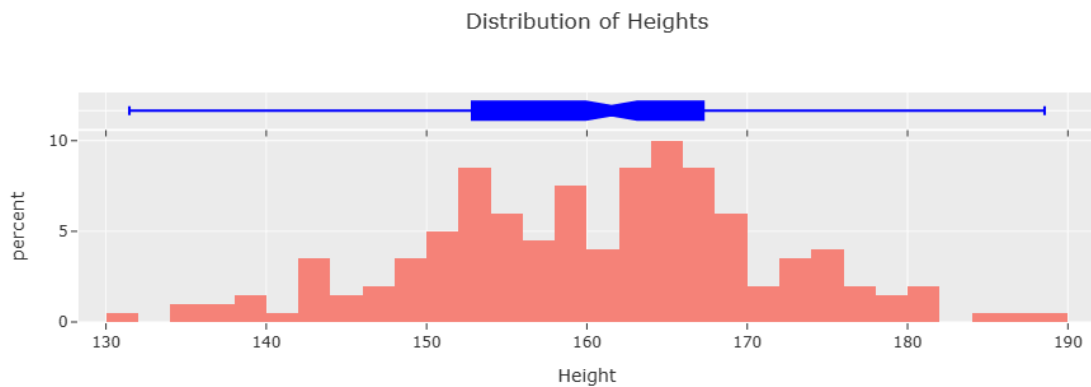
A histogram is used to represent continuous data in the form of bar. Each bar has discrete values in bar graphs, whereas in histograms, we have bars representing a range of values. Histograms show frequency distributions. We can use histograms to visualize:

- Students marks distribution
- Frequency of waiting time of customers in a Bank

```
In [12]: ##Example 4: Let us illustrate the distribution of heights of 200 people using a histogram
import numpy as np

#Here we will concentrate on heights which are 160 and the standard deviation is 11
heights_array = np.random.normal(160, 11, 200)

## Use plotly express histogram chart function px.histogram. Provide input data x to
fig = px.histogram(x=heights_array,
                   labels=dict(x="Height"),
                   template="ggplot2",
                   opacity=0.9,
                   nbins=30,
                   histnorm="percent",
                   title="Distribution of Heights",
                   marginal="box")
fig.update_traces(fillcolor="blue", marker=dict(color="blue"), selector=dict(type="area"))
fig.show()
```



Inferences

From this we can analyze that there are around 2 people who are at the height of 130cm and 45 people at the height of 160 cm

5. Bubble Plot:

A bubble plot is used to show the relationship between 3 or more variables. It is an extension of a scatter plot. Bubble plots are ideal for visualizing:

- Global Economic position of Industries
- Impact of viruses on Diseases

```
In [13]: ##Example 4: Let us illustrate crime statistics of US cities with a bubble chart

#Create a dictionary having city,numberofcrimes and year as 3 keys
crime_details = {
    'City' : ['Chicago', 'Chicago', 'Austin', 'Austin','Seattle','Seattle'],
    'Numberofcrimes' : [1000, 1200, 400, 700,350,1500],
    'Year' : ['2007', '2008', '2007', '2008','2007','2008'],
}

# create a Dataframe object with the dictionary
df = pd.DataFrame(crime_details)

df
```

Out[13]:

	City	Numberofcrimes	Year
0	Chicago	1000	2007
1	Chicago	1200	2008
2	Austin	400	2007
3	Austin	700	2008
4	Seattle	350	2007
5	Seattle	1500	2008

In [14]: *## Group the number of crimes by city and find the total number of crimes per city*
 bub_data = df.groupby('City')['Numberofcrimes'].sum().reset_index()

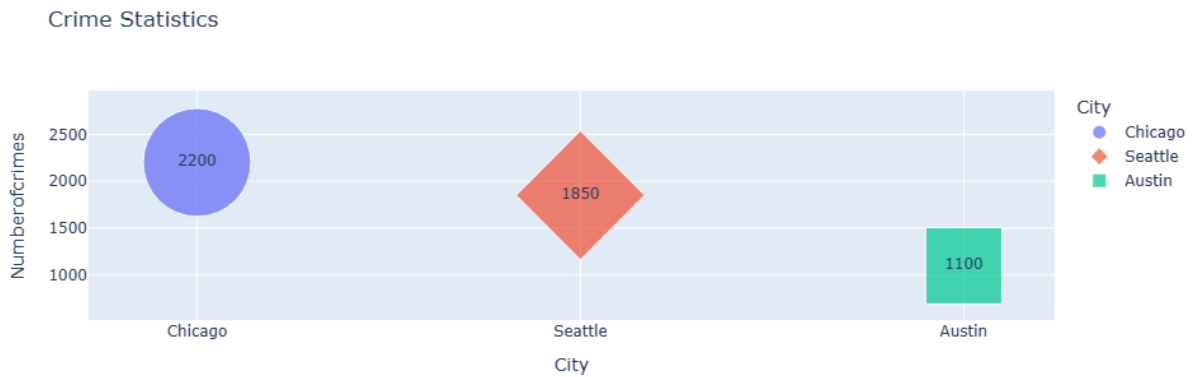
In [15]: *##Display the grouped dataframe*
 bub_data

Out[15]:

	City	Numberofcrimes
0	Austin	1100
1	Chicago	2200
2	Seattle	1850

In [16]: *## Bubble chart using px.scatter function with x ,y and size variables defined.Title*
 fig = px.scatter(data_frame=bub_data,
 x="City",
 category_orders={"City":["Chicago", "Seattle", "Au
 y="Numberofcrimes",
 text="Numberofcrimes",
 color="City",
 hover_name="City",
 title='Crime Statistics',

 symbol="City",
 size="Numberofcrimes",
 size_max=60,
 trendline="ols",
 # marginal_x="hist",
 # marginal_y="violin"
)
 fig.show()



Inferences

The size of the bubble in the bubble chart indicates that Chicago has the highest crime rate when compared with the other 2 cities.

6.Pie Plot:

A pie plot is a circle chart mainly used to represent proportion of part of given data with respect to the whole data. Each slice represents a proportion and on total of the proportion becomes a whole. We can use bar plots in visualizing:

- Sales turnover percentatge with respect to different products
- Monthly expenditure of a Family

```
In [17]: ## Monthly expenditure of a family
# Random Data
exp_percent= [20, 50, 10,8,12]
house_holdcategories = ['Grocery', 'Rent', 'School Fees','Transport','Savings']
```

```
In [18]: # Use px.pie function to create the chart. Input dataset.
# Values parameter will set values associated to the sector. 'exp_percent' feature
# Labels for the sector are passed to the `house hold categoris` parameter.
fig = px.pie(values=exp_percent, names=house_holdcategories, hole=0.6, title='House
fig.update_layout(paper_bgcolor="rgb(255, 255, 255)", width=400)
fig.show()
```

Household Expenditure



Inferences

From this pie chart we can find that the family expenditure is maximum for rent.

7.Sunburst Charts:

Sunburst charts represent hierarchial data in the form of concentric circles. Here the innermost circle is the root node which defines the parent, and then the outer rings move down the hierarchy from the centre. They are also called radial charts. We can use them to plot

- Worldwide mobile Sales where we can drill down as follows:
 - innermost circle represents total sales
 - first outer circle represents continentwise sales
 - second outer circle represents countrywise sales within each continent
- Disease outbreak hierarchy
- Real Estate Industrial chain

```
In [19]: ##Example 4: Let us illustrate plot the
#Create a dictionary having a set of people represented by a character array and th
## array and the values are the values associated to the vectors.
data = dict(
    character=["Eve", "Cain", "Seth", "Enos", "Noam", "Abel", "Awan", "Enoch", "Azura"],
    parent=["", "Eve", "Eve", "Seth", "Seth", "Eve", "Eve", "Awan", "Eve"],
    value=[10, 14, 12, 10, 2, 6, 6, 4, 4])

fig = px.sunburst(
    data,
    names='character',
    parents='parent',
    values='value',
    title="Family chart"
)
```

```
fig.update_layout(width=400)
fig.show()
```

Family chart



Inferences

It is found that here the innermost circle **Eve** represents the parent and the second outer circle represents his children **Cain, Seth** and so on. Further the outermost circle represents his grandchildren **Enoch** and **Enos**

II- Practice Exercises: Apply your Plotly Skills to an Airline Dataset

The Reporting Carrier On-Time Performance Dataset contains information on approximately 200 million domestic US flights reported to the United States Bureau of Transportation Statistics. The dataset contains basic information about each flight (such as date, time, departure airport, arrival airport) and, if applicable, the amount of time the flight was delayed and information about the reason for the delay. This dataset can be used to predict the likelihood of a flight arriving on time.

Preview data, dataset metadata, and data glossary [here](#).

Read Data

```
In [20]: # Read the airline data into pandas dataframe
airline_data = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-storage.app
                        encoding = "ISO-8859-1",
                        dtype={'Div1Airport': str, 'Div1TailNum': str,
                              'Div2Airport': str, 'Div2TailNum': str})
```

```
In [21]: # Preview the first 5 lines of the loaded data
airline_data.head()
```

Out[21]:

	Unnamed: 0	Year	Quarter	Month	DayofMonth	DayOfWeek	FlightDate	Reporting_Air
0	1295781	1998	2	4	2	4	1998-04-02	
1	1125375	2013	2	5	13	1	2013-05-13	
2	118824	1993	3	9	25	6	1993-09-25	
3	634825	1994	4	11	12	6	1994-11-12	
4	1888125	2017	3	8	17	4	2017-08-17	

5 rows × 110 columns



In [22]:

airline_data

Out[22]:

	Unnamed: 0	Year	Quarter	Month	DayofMonth	DayOfWeek	FlightDate	Reporting
0	1295781	1998	2	4	2	4	1998-04-02	
1	1125375	2013	2	5	13	1	2013-05-13	
2	118824	1993	3	9	25	6	1993-09-25	
3	634825	1994	4	11	12	6	1994-11-12	
4	1888125	2017	3	8	17	4	2017-08-17	
...	...	...	...	...	...	...	...	
26995	821542	2017	1	1	24	2	2017-01-24	
26996	1910565	2013	2	6	27	4	2013-06-27	
26997	9055	2016	3	8	26	5	2016-08-26	
26998	84136	2009	3	8	8	6	2009-08-08	
26999	113029	1993	3	7	17	6	1993-07-17	

27000 rows × 110 columns

In [23]:

```
# Shape of the data
airline_data.shape
```

Out[23]: (27000, 110)

In [24]:

```
# Randomly sample 500 data points. Setting the random state to be 42 so that we get
data = airline_data.sample(n=500, random_state=42, replace=False)
```

In [25]:

```
# Get the shape of the trimmed data
data.shape
```

Out[25]: (500, 110)

In [26]:

```
data.info(verbose=False)
```

```
<class 'pandas.DataFrame'>
Index: 500 entries, 5312 to 16705
Columns: 110 entries, Unnamed: 0 to Div5TailNum
dtypes: float64(72), int64(19), str(19)
memory usage: 478.4 KB
```

It would be interesting if we visually capture details such as

- Departure time changes with respect to airport distance.
- Average Flight Delay time over the months
- Comparing number of flights in each destination state
- Number of flights per reporting airline
- Distrubution of arrival delay
- Proportion of distance group by month (month indicated by numbers)
- Hierarchical view in the order of month and destination state holding value of number of flights

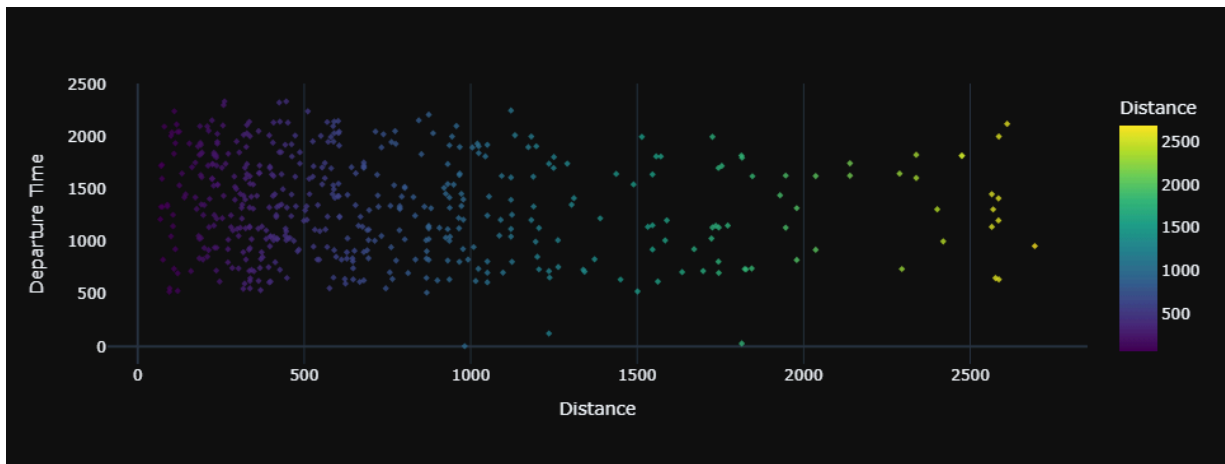
1. Scatter Plot

Let us use a scatter plot to represent departure time changes with respect to airport distance

This plot should contain the following

- Title as **Distance vs Departure Time**.
- x-axis label should be **Distance**
- y-axis label should be **DeptTime**
- **Distance** column data from the flight delay dataset should be considered in x-axis
- **DepTime** column data from the flight delay dataset should be considered in y-axis
- Scatter plot markers should be of red color

```
In [27]: ## Write your code here
fig = px.scatter(
    data_frame=data,
    x="Distance",
    y="DepTime",
    color="Distance",
    color_continuous_scale="viridis",
    labels=dict(DepTime="Departure Time"),
    template="plotly_dark"
)
fig.update_traces(marker=dict(size=4, symbol="diamond", opacity=0.8)).update_layout
fig.show()
```



Inferences

It can be inferred that there are more flights round the clock for shorter distances. However, for longer distance there are limited flights through the day.

2. Line Plot

Let us now use a line plot to extract average monthly arrival delay time and see how it changes over the year.

This plot should contain the following

- Title as **Month vs Average Flight Delay Time.**
- x-axis label should be **Month**
- y-axis label should be **ArrDelay**
- A new dataframe **line_data** should be created which consists of 2 columns average **arrival delay time per month** and **month** from the dataset
- **Month** column data from the line_data dataframe should be considered in x-axis
- **ArrDelay** column data from the ine_data dataframeshould be considered in y-axis
- Plotted line in the line plot should be of green color

```
In [28]: # Group the data by Month and compute average over arrival delay time.
line_data = data.groupby('Month')['ArrDelay'].mean().reset_index()
```

```
In [29]: # Display the data
line_data
```

Out[29]:

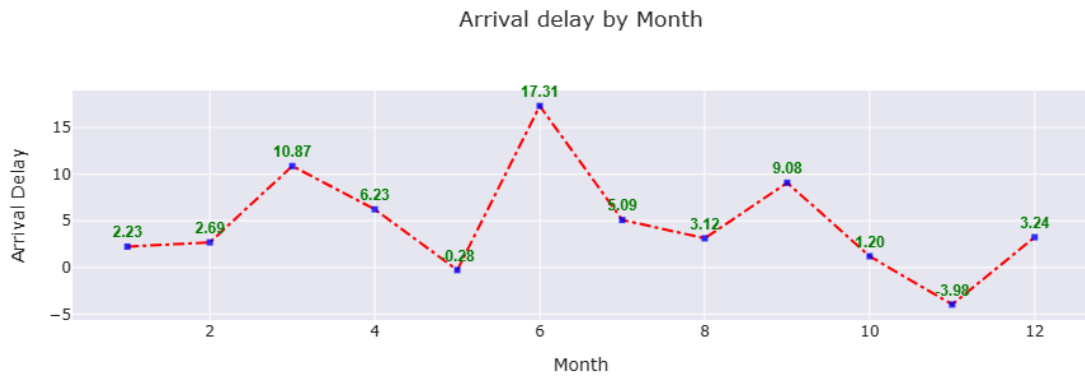
	Month	ArrDelay
0	1	2.232558
1	2	2.687500
2	3	10.868421
3	4	6.229167
4	5	-0.279070
5	6	17.310345
6	7	5.088889
7	8	3.121951
8	9	9.081081
9	10	1.200000
10	11	-3.975000
11	12	3.240741

In [30]: *## Write your code here*

```

fig = px.line(data_frame=line_data,
               x="Month",
               y="ArrDelay",
               text="ArrDelay",
               markers=True,
               title="Arrival delay by Month",
               hover_data={"ArrDelay":":.2f"},
               labels={"ArrDelay": "Arrival Delay"},
               template="seaborn"
            )
fig.update_traces(line=dict(color="red", width=2, dash="dashdot"),
                  marker=dict(symbol="square", color="blue", size=5),
                  textposition="top center",
                  cliponaxis=False,
                  textfont=dict(family="arial", weight="bold", size=12),
                  texttemplate="%{y:.2f}"
                  )
fig.show()

```

Inferences

It is found that in the month of June the average monthly delay time is the maximum

3. Bar Chart

Let us use a bar chart to extract number of flights that goes to a destination

This plot should contain the following

- Title as **Total number of flights to the destination state split by reporting air.**
- x-axis label should be **DestState**
- y-axis label should be **Flights**
- Create a new dataframe called **bar_data** which contains 2 columns **DestState** and **Flights**. Here **flights** indicate total number of flights in each combination.

```
In [31]: # Group the data by destination state and reporting airline. Compute total number of flights
bar_data = data.groupby(['DestState'])['Flights'].sum().reset_index()
```

```
In [32]: # Display the data
bar_data.head()
```

```
Out[32]:
```

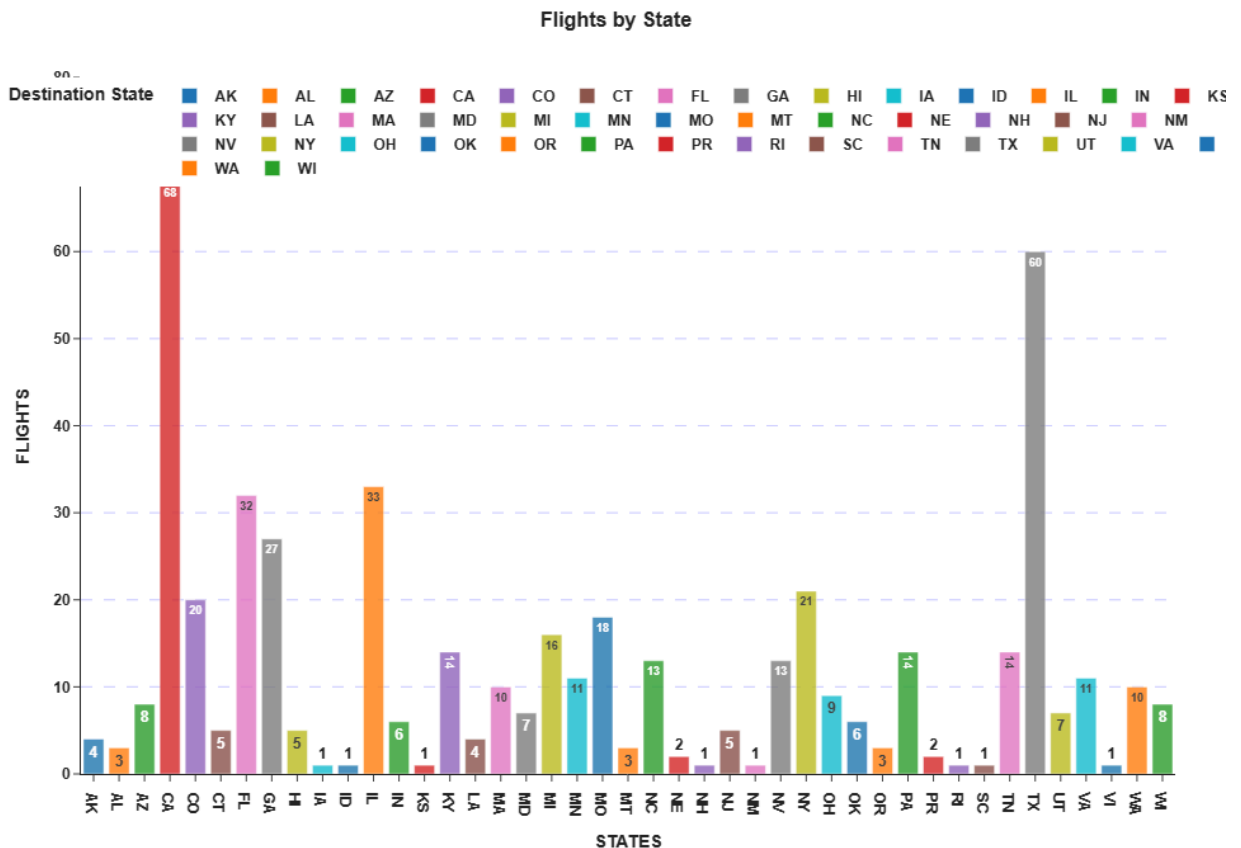
	DestState	Flights
0	AK	4.0
1	AL	3.0
2	AZ	8.0
3	CA	68.0
4	CO	20.0

```
In [33]: ## Write your code here
fig = px.bar(data_frame=bar_data,
              x="DestState",
```

```

y="Flights",
range_y=(0, 80),
text="Flights",
color="DestState",
opacity=0.8,
template="simple_white",
labels={"DestState": "Destination State"}
)
fig.update_layout(font=dict(family="arial", size=12, weight="bold"),
                    title=dict(text="Flights by State", x=0.5, xanchor="center",
                                xaxis=dict(title="STATES", showgrid=False),
                                yaxis=dict(title="FLIGHTS", showgrid=True, gridwidth=1,
                                            legend=dict(orientation="h", x=0, y=1, xanchor="left",
                                                        # plot_bgcolor="gray",
                                                        # paper_bgcolor="white",
                                                        # hovermode="x unified",
                                                        margin=dict(l=40, r=40, t=80, b=40),
                                                        width=900,
                                                        height=700
                                            )
                                )
                    )
fig.update_traces(cliponaxis=False).show()

```



Inferences

It is found that maximum flights are to destination state **CA** which is around 68 and there is only 1 flight to destination state **VT**

4. Histogram

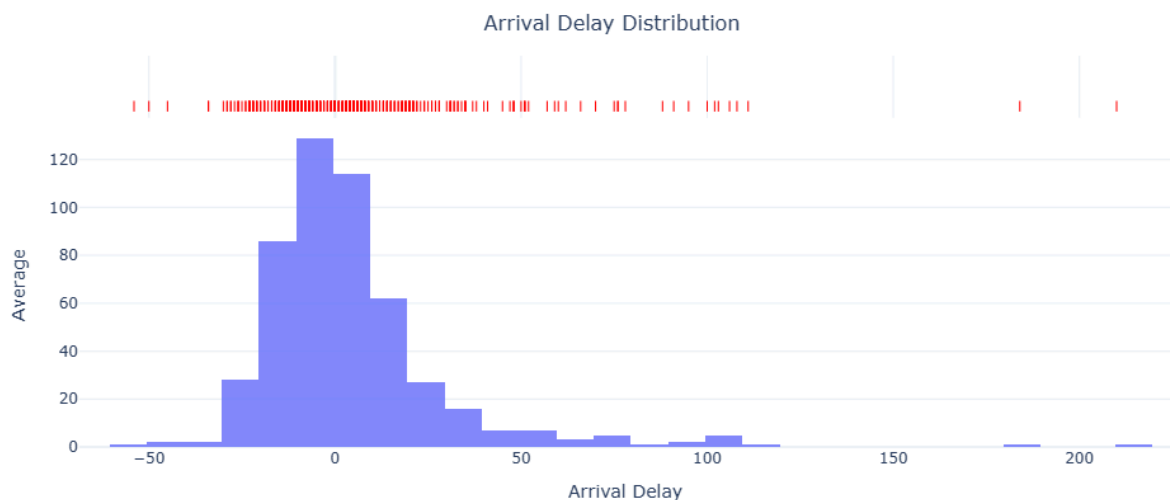
Let us represent the distribution of arrival delay using a histogram

This plot should contain the following

- Title as **Total number of flights to the destination state split by reporting air.**
- x-axis label should be **ArrayDelay**
- y-axis will show the count of arrival delay

```
In [34]: # Set missing values to 0
data['ArrDelay'] = data['ArrDelay'].fillna(0)
```

```
In [35]: ## Write your code here
fig = px.histogram(data_frame=data,
                   x="ArrDelay",
                   template="plotly_white",
                   opacity=0.8,
                   labels={"ArrDelay": "Arrival Delay"},
                   nbins=50,
                   histfunc="avg",
                   marginal="rug")
fig.update_traces(marker=dict(color="red"), selector=dict(type="box"))
fig.update_layout(
    paper_bgcolor="white",
    margin=dict(r=40, l=40, t=40, b=40),
    height=400,
    width=800,
    title=dict(text="Arrival Delay Distribution", x=0.5, xanchor="center", font
    ),
    yaxis=dict(title=dict(text="Average"))
)
fig.show()
```



Inferences

It is found that there is only max of 5 flights with an arrival delay of 50-54 minutes and around 17 flights with an arrival delay of 20-25 minutes

5. Bubble Plot

Let use a bubble plot to represent number of flights as per reporting airline

This plot should contain the following

- Title as **Reporting Airline vs Number of Flights**.
- x-axis label should be **Reporting_Airline**
- y-axis label should be **Flights**
- size of the bubble should be **Flights** indicating number of flights
- Name of the hover tooltip to `reporting_airline` using `hover_name` parameter.

```
In [36]: # Group the data by reporting airline and get number of flights
bub_data = data.groupby('Reporting_Airline')['Flights'].sum().reset_index()
```

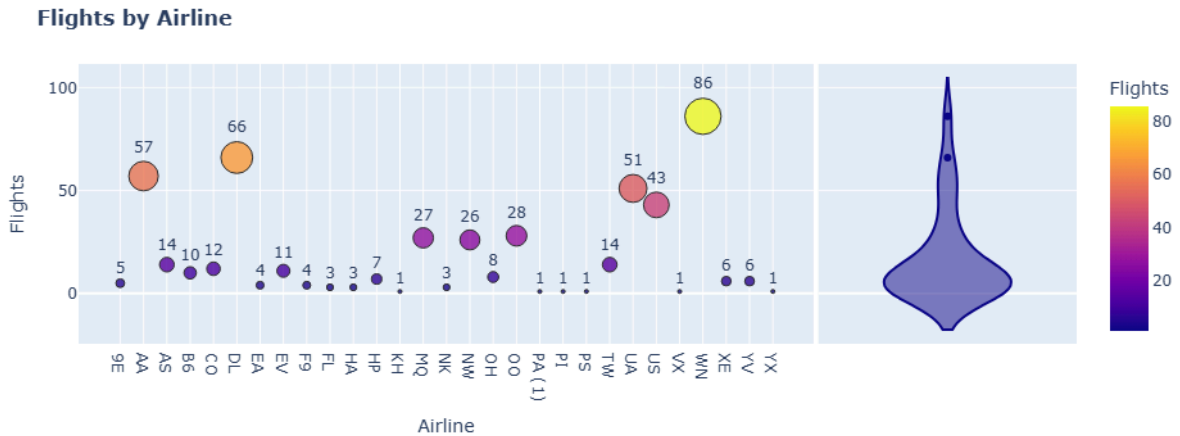
```
In [37]: bub_data.head()
```

```
Out[37]:
```

	Reporting_Airline	Flights
0	9E	5.0
1	AA	57.0
2	AS	14.0
3	B6	10.0
4	CO	12.0

```
In [38]: ## Write your code here
fig = px.scatter(data_frame=bub_data,
                 x="Reporting_Airline",
                 y="Flights",
                 color="Flights",
                 # color_continuous_scale="viridis",
                 text="Flights",
                 labels={"Reporting_Airline": "Airline"},
                 size="Flights",
                 # symbol="Flights",
                 marginal_y="violin"
                )
fig.update_traces(marker=dict(colorscale="viridis", showscale=True, opacity=0.8),
                  marker_line=dict(color="black", width=1),
                  textposition="top center",
                  cliponaxis=False,
                  selector=dict(type="scatter"),
                 )
```

```
fig.update_layout(title=dict(text="Flights by Airline",
                             font=dict(size=16, weight=
fig.show()
```



Inferences

It is found that the reporting airline **WN** has the highest number of flights which is around 86

6. Pie Chart

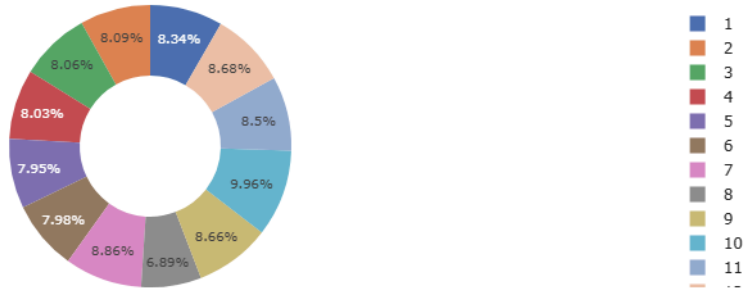
Let us represent the proportion of distance group by month (month indicated by numbers)

This plot should contain the following

- Title as **Distance group proportion by month.**
- values should be **Month**
- names should be **DistanceGroup**

```
In [39]: ## Write your code here
fig = px.pie(data_frame=data.groupby('Month')['DistanceGroup'].mean().reset_index(),
             template="seaborn",
             names="Month",
             values="DistanceGroup",
             labels={"DistanceGroup": "Distance Group"},
             hole=0.5)
fig.update_traces(textfont=dict(size=10), sort=False)
fig.update_layout(width=400, title=dict(text="Distance Group by Month"))
fig.show()
```

Distance Group by Month



Inferences

It is found that February month has the highest distance group proportion

7. SunBurst Charts

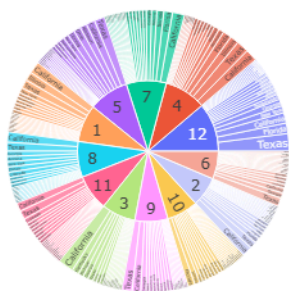
Let us represent the hierarchical view in othe order of month and destination state holding value of number of flights

This plot should contain the following

- Define hierarchy of sectors from root to leaves in `path` parameter. Here, we go from `Month` to `DestStateName` feature.
- Set sector values in `values` parameter. Here, we can pass in `Flights` feature.
- Show the figure.
- Title as **Flight Distribution Hierarchy**

```
In [40]: ## Write your code here
fig = px.sunburst(
    data_frame=data,
    path=["Month", "DestStateName"],
    values="Flights",
    # maxdepth=2
)
fig.update_layout(width=400, title=dict(text="Flights by Month and State"))
fig.show()
```

Flights by Month and State



Inferences

Here the **Month** numbers present in the innermost concentric circle is the root and for each month we will check the **number of flights** for the different **destination states** under it.

Summary

Congratulations for completing your lab.

In this lab, you have learnt how to use `plotly.graph_objects` and `plotly.express` for creating plots and charts.

Author(s)

Saishruthi Swaminathan

Lakshmi Holla

Other Contributor(s)

Lavanya T S

© IBM Corporation 2023. All rights reserved.

{toggle}##

{toggle}|

{toggle}|

{toggle}|

